Database Systems(CSL3050)

Lab Assignment 6 - Report

Soumen Kumar
Roll No-B22ES006

1 Introduction

This report contains descriptions and results of various PL/SQL queries executed on a database. Each query addresses different aspects of PL-SQL, such as data retrieval, aggregation, and ranking. The screenshots of the query outputs are provided for clarity.

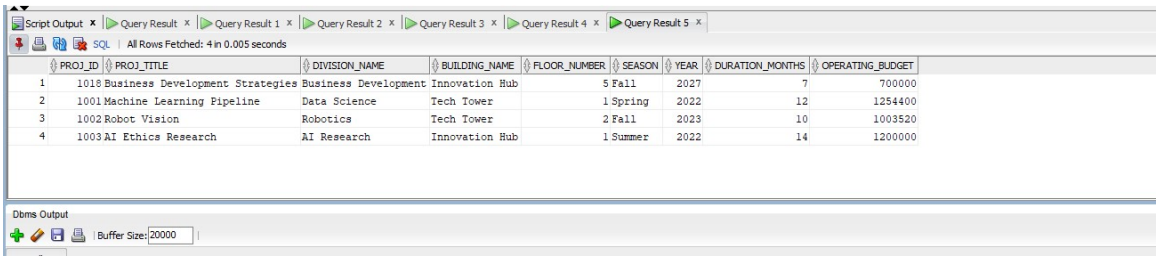
Query 1: Merge Tables

Description:
This PL/SQL block merges the `PROJECT` and `DIVISION` tables with the `ASSIGNMENT` table to reduce redundancy by matching `proj_id`, `division_name`, and `building_name`.

Code:

```
CREATE TABLE merged_project_assignment (
    proj_id NUMBER PRIMARY KEY,
    proj_title VARCHAR2(100),
    division_name VARCHAR2(50),
    building_name VARCHAR2(50),
    floor_number NUMBER,
    season VARCHAR2(20),
    year NUMBER,
    duration_months NUMBER,
    operating_budget NUMBER
);
BEGIN
    INSERT INTO merged_project_assignment (proj_id, proj_title, division_name, building_name, floor_number
    , season, year, duration_months, operating_budget)
    SELECT p.proj_id, p.proj_title, p.division_name, a.building_name, a.floor_number, a.season, a.year, p.
    duration_months, d.operating_budget
    FROM project p
    JOIN assignment a ON p.proj_id = a.proj_id
    JOIN division d ON p.division_name = d.division_name AND a.building_name = d.building_name;
    COMMIT;
END;
/
select * from merged_project_assignment;
```

Diagram:



	PROJ_ID	PROJ_TITLE	DIVISION_NAME	BUILDING_NAME	FLOOR_NUMBER	SEASON	YEAR	DURATION_MONTHS	OPERATING_BUDGET
1	1018	Business Development Strategies	Business Development	Innovation Hub	5	Fall	2027	7	700000
2	1001	Machine Learning Pipeline	Data Science	Tech Tower	1	Spring	2022	12	1254400
3	1002	Robot Vision	Robotics	Tech Tower	2	Fall	2023	10	1003520
4	1003	AI Ethics Research	AI Research	Innovation Hub	1	Summer	2022	14	1200000

Figure 1: Table Merge Visualization

Query 2: Output Employee Details

Description:
This PL/SQL block outputs the employee name, division name, and project title for employees working on projects with a duration longer than 12 months.

Code:

```
BEGIN
    FOR rec IN (
        SELECT emp.emp_name, emp.division_name, p.proj_title
        FROM employee emp
        JOIN works_on w ON emp.ID = w.ID
        JOIN project p ON w.proj_id = p.proj_id
        WHERE p.duration_months > 12
    ) LOOP
        DBMS_OUTPUT.PUT_LINE('Employee Name: ' || rec.emp_name);
        DBMS_OUTPUT.PUT_LINE('Division Name: ' || rec.division_name);
        DBMS_OUTPUT.PUT_LINE('Project Title: ' || rec.proj_title);
        DBMS_OUTPUT.PUT_LINE('-------------------------------------');
    END LOOP;
END;
/
```

Diagram:

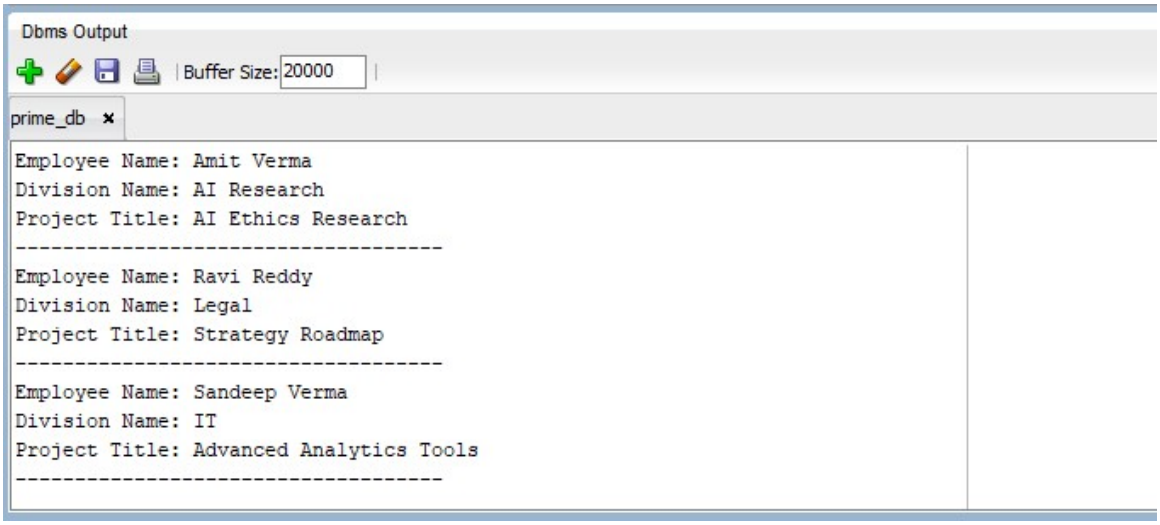


Figure 2: Employee Details Output

Query 3: Trainees with Mentors

Description:
This PL/SQL block finds the names of trainees who have mentors from the same division.

Code:

```
BEGIN
    FOR rec IN (
        SELECT t.trainee_name, e.emp_name AS mentor_name, t.division_name
        FROM trainee t
        JOIN mentor m ON t.ID = m.trainee_ID
        JOIN employee e ON m.mentor_ID = e.ID
        WHERE t.division_name = e.division_name
    ) LOOP
        DBMS_OUTPUT.PUT_LINE('Trainee Name: ' || rec.trainee_name);
        DBMS_OUTPUT.PUT_LINE('Mentor Name: ' || rec.mentor_name);
        DBMS_OUTPUT.PUT_LINE('Division: ' || rec.division_name);
        DBMS_OUTPUT.PUT_LINE('-------------------------------------');
    END LOOP;
END;
/
```

Diagram:

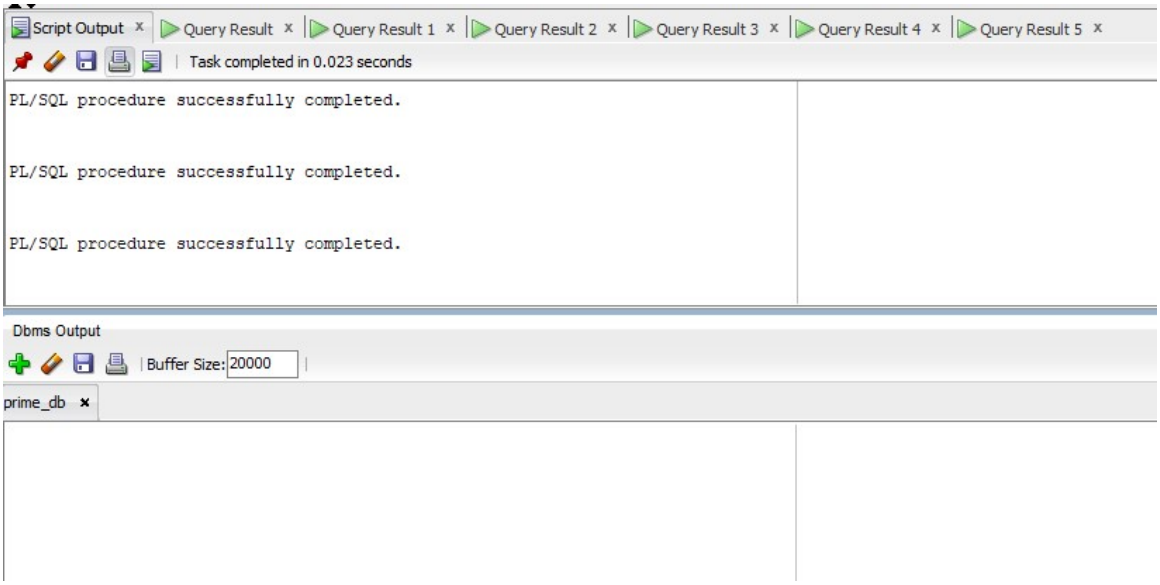


Figure 3: Trainees with Same Division Mentors

Query 4: Highest Operating Budget

Description:
This PL/SQL block finds the division name with the highest operating budget.

Code:

```
DECLARE
    v_division_name division.division_name%TYPE;
    v_highest_budget division.operating_budget%TYPE;
BEGIN
    -- Select the division with the highest operating budget
    SELECT division_name, operating_budget
    INTO v_division_name, v_highest_budget
    FROM division
    WHERE operating_budget = (SELECT MAX(operating_budget) FROM division);

    -- Output the division name and the highest budget
    DBMS_OUTPUT.PUT_LINE('Division with the highest operating budget: ' || v_division_name);
    DBMS_OUTPUT.PUT_LINE('Operating Budget: ' || v_highest_budget);
END;
/
```

Diagram:

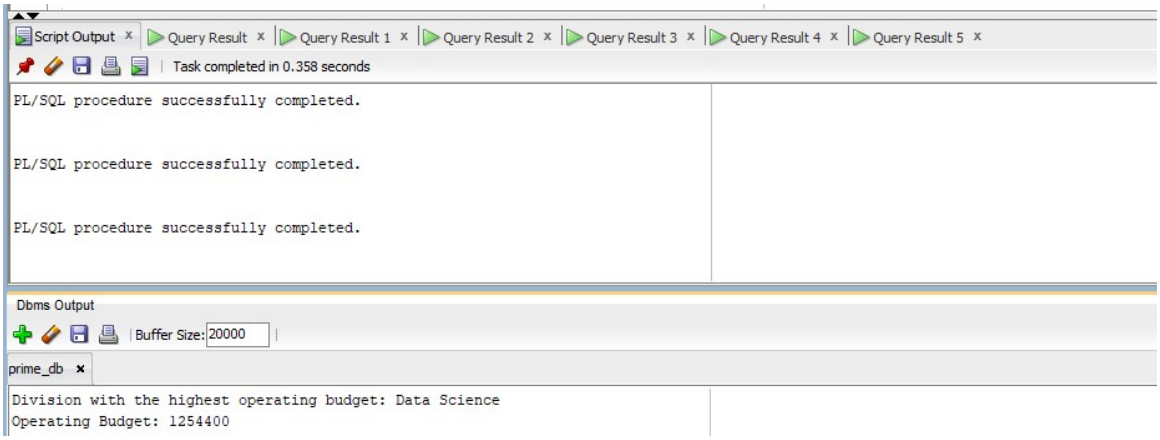


Figure 4: Division with Highest Operating Budget

Query 5: Validate Slot Availability

Description:
This PL/SQL block checks the validity of the availability entries. A valid slot should have `start_time` before `end_time` and should not overlap with other entries.

Code:

```
DECLARE
    CURSOR availability_cursor IS
        SELECT slot_id, weekday, start_time, end_time
        FROM availability;

    v_slot_id availability.slot_id%TYPE;
    v_weekday availability.weekday%TYPE;
    v_start_time availability.start_time%TYPE;
    v_end_time availability.end_time%TYPE;

    slot_overlap EXCEPTION;
BEGIN
    -- Loop through each availability entry
    FOR avail_rec IN availability_cursor LOOP
        v_slot_id := avail_rec.slot_id;
        v_weekday := avail_rec.weekday;
        v_start_time := avail_rec.start_time;
        v_end_time := avail_rec.end_time;

        -- Check if start_time is before end_time
        IF v_start_time >= v_end_time THEN
            DBMS_OUTPUT.PUT_LINE('Error: Invalid entry for slot_id ' || v_slot_id || ' on ' || v_weekday
    || ': start_time is not before end_time.');
        END IF;

        -- Check for overlapping slots with the same slot_id and weekday
        DECLARE
            v_count INTEGER;
        BEGIN
            SELECT COUNT(*)
            INTO v_count
            FROM availability a
            WHERE a.slot_id = v_slot_id
              AND a.weekday = v_weekday
              AND (a.start_time < v_end_time AND a.end_time > v_start_time)
              AND (a.slot_id <> avail_rec.slot_id OR a.start_time <> avail_rec.start_time OR a.end_time <>
     avail_rec.end_time);

            IF v_count > 0 THEN
                DBMS_OUTPUT.PUT_LINE('Error: Overlapping slot detected for slot_id ' || v_slot_id || ' on
    ' || v_weekday);
            END IF;
        END;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Validation completed.');
EXCEPTION
    WHEN slot_overlap THEN
        DBMS_OUTPUT.PUT_LINE('Error: Slot overlap detected.');
END;
/
```

Diagram:

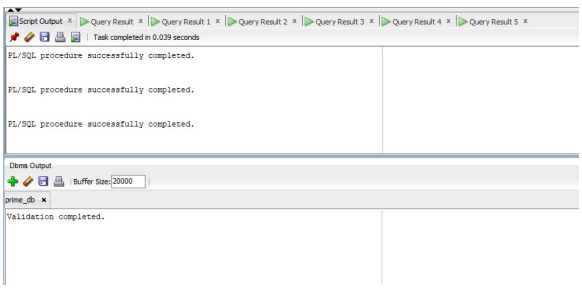


Figure 5: Slot Availability Validation

Query 6: Count Projects by Season

Description:
This PL/SQL block counts the number of projects each division is working on during a specified season.

Code:

```
DECLARE
    v_season VARCHAR2(20) := 'Winter';  -- Specify the season to filter
    CURSOR project_cursor IS
        SELECT p.division_name, COUNT(a.proj_id) AS project_count
        FROM assignment a
        JOIN project p ON a.proj_id = p.proj_id
        WHERE a.season = v_season
        GROUP BY p.division_name;

    v_division_name project.division_name%TYPE;
    v_project_count NUMBER;
BEGIN
    -- Loop through all projects for the specified season
    FOR proj_rec IN project_cursor LOOP
        v_division_name := proj_rec.division_name;
        v_project_count := proj_rec.project_count;

        -- Output the result
        DBMS_OUTPUT.PUT_LINE('Division: ' || v_division_name || ', Projects: ' || v_project_count);
    END LOOP;

    -- If no projects were found for the season
    IF project_cursor%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('No projects found for the season: ' || v_season);
    END IF;

    DBMS_OUTPUT.PUT_LINE('Project count for season ' || v_season || ' completed.');
END;
/
```

Diagram:

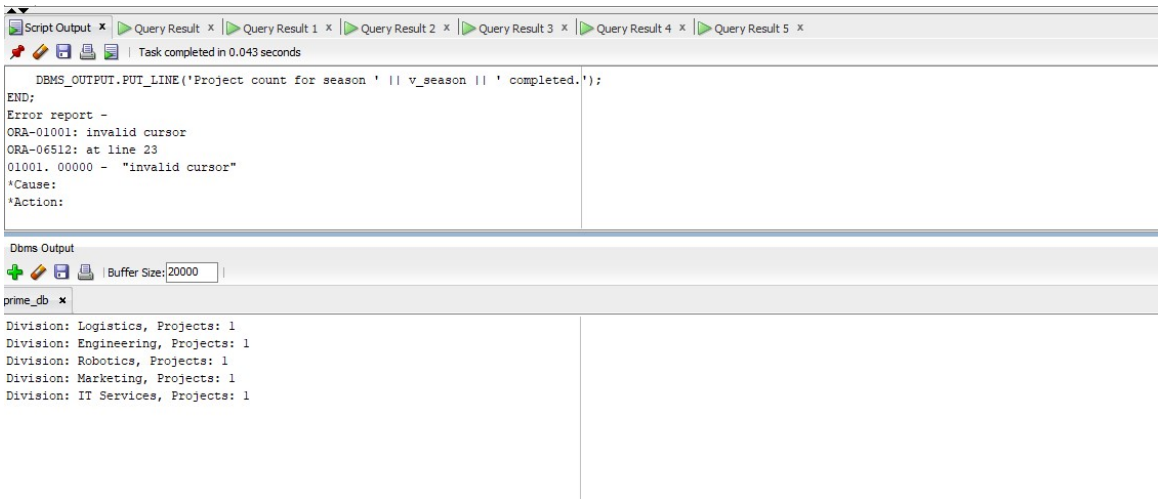


Figure 6: Count of Projects by Season

Query 7: Update Project IDs

Description:
This PL/SQL block updates the `proj_id` in the `works_on` table to follow a specific format.

Code:

```
--updating the proj_id column to VARCHAR2 in works_on and project tables but not able to update type due to some reference constraint
--so creating a new col and storing req projectid in it
ALTER TABLE works_on ADD formatted_proj_id VARCHAR2(50);

DECLARE
    CURSOR proj_cursor IS
        SELECT w.ID, w.proj_id, p.division_name
        FROM works_on w
        JOIN project p ON w.proj_id = p.proj_id;

    v_new_proj_id VARCHAR2(50);
BEGIN
    -- Loop through each project in works_on
    FOR proj_rec IN proj_cursor LOOP
        -- Generate the new proj_id in the format [First 3 letters of division_name]_[proj_id]
        v_new_proj_id := SUBSTR(proj_rec.division_name, 1, 3) || '_' || proj_rec.proj_id;

        -- Update the formatted_proj_id in the works_on table
        UPDATE works_on
        SET formatted_proj_id = v_new_proj_id
        WHERE ID = proj_rec.ID
        AND proj_id = proj_rec.proj_id;

        -- Output the updated formatted project ID
        DBMS_OUTPUT.PUT_LINE('Updated formatted_proj_id to: ' || v_new_proj_id || ' for employee ID: ' || proj_rec.ID);
    END LOOP;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('All formatted_proj_ids updated successfully.');
END;
/
```

Diagram:

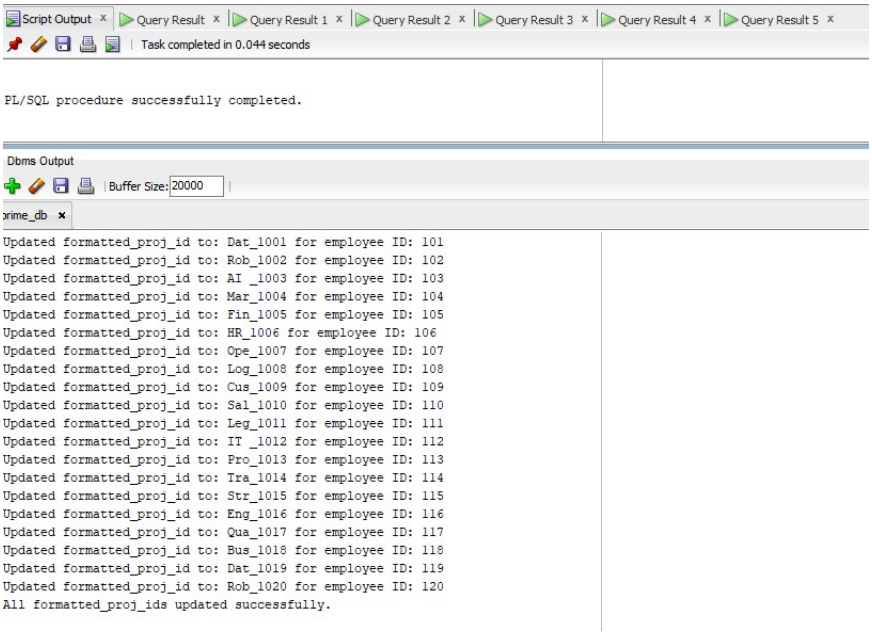


Figure 7: Update of Project IDs

Query 8: Calculate Total Salary

Description:
This PL/SQL block computes the total salary expenditure for a project based on employees assigned to that project.

Code:

```
DECLARE
    v_proj_id NUMBER := 1005;
    v_total_salary NUMBER := 0;
BEGIN
    -- Loop through all employees working on the given project
    FOR emp_rec IN (
        SELECT e.emp_name, e.annual_salary
        FROM employee e
        JOIN works_on w ON e.ID = w.ID
        WHERE w.proj_id = v_proj_id
    ) LOOP
        -- Add each employee's annual salary to the total salary
        v_total_salary := v_total_salary + emp_rec.annual_salary;
    END LOOP;

    -- Output the total salary for the project
    DBMS_OUTPUT.PUT_LINE('Total Salary Expenditure for Project ' || v_proj_id || ': ' || v_total_salary);
END;
/
```

Diagram:

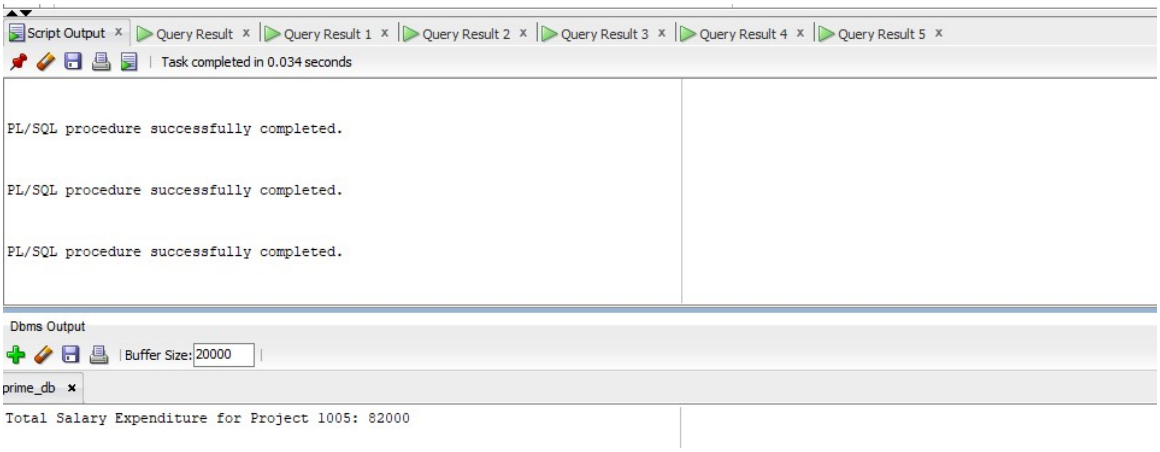


Figure 8: Total Salary Expenditure Calculation

Query 9: List Employees on Summer Projects

Description:
This PL/SQL block outputs the first name, last name, and division name of employees assigned to a project during the "Summer" season for the current year.

Code:

```
DECLARE
    v_current_year NUMBER := EXTRACT(YEAR FROM SYSDATE);
BEGIN
    FOR emp_rec IN (
        SELECT SUBSTR(e.emp_name, 1, INSTR(e.emp_name, ' ') - 1) AS first_name,  -- Extract first name
               SUBSTR(e.emp_name, INSTR(e.emp_name, ' ') + 1) AS last_name,       -- Extract last name
               e.division_name
        FROM employee e
        JOIN works_on w ON e.ID = w.ID
        JOIN project p ON w.proj_id = p.proj_id
        WHERE w.season = 'Summer'
          AND w.year = v_current_year
    ) LOOP
        -- Output the employee details
        DBMS_OUTPUT.PUT_LINE('First Name: ' || emp_rec.first_name || ', Last Name: ' || emp_rec.last_name
    || ', Division: ' || emp_rec.division_name);
    END LOOP;
END;
/
```

Diagram:

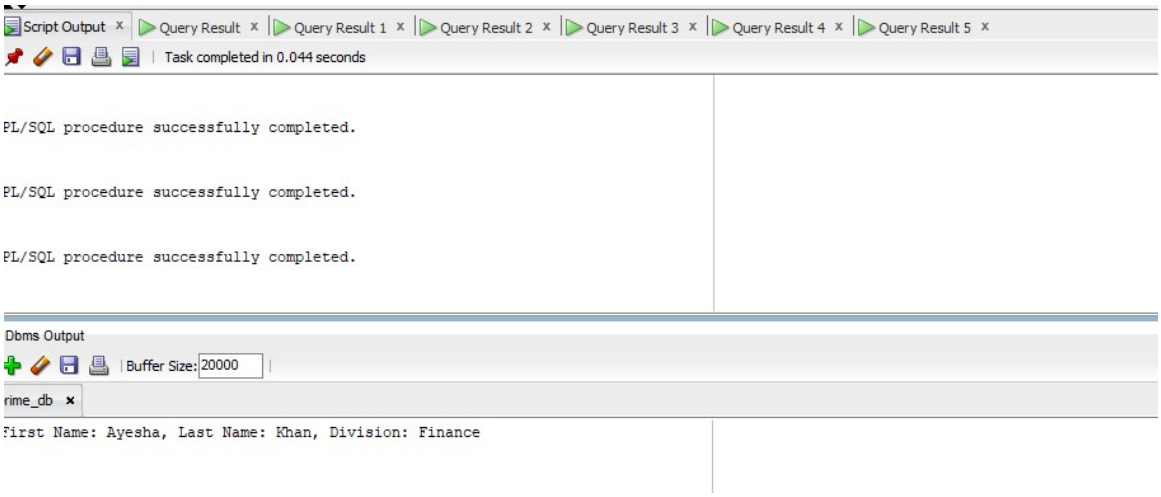


Figure 9: Employees on Summer Projects

Query 10: Most Frequent Division in Projects

Description:
This PL/SQL block finds the division that has the most number of projects assigned to it and lists the division name and project title.

Code:

```
DECLARE
    v_max_projects NUMBER;  -- To store the maximum number of projects
    v_division_name VARCHAR2(50);  -- To store the division name with the most projects
BEGIN
    -- Get the division with the most number of projects
    SELECT division_name, COUNT(proj_id) AS project_count
    INTO v_division_name, v_max_projects
    FROM project
    GROUP BY division_name
    ORDER BY project_count DESC
    FETCH FIRST ROW ONLY;  -- This will get the division with the most projects

    -- Output the results
    DBMS_OUTPUT.PUT_LINE('Division with the most projects: ' || v_division_name || ' with ' ||
    v_max_projects || ' projects.');

    -- List the project titles for the division with the most projects
    FOR proj_rec IN (
        SELECT proj_title
        FROM project
        WHERE division_name = v_division_name
    ) LOOP
        DBMS_OUTPUT.PUT_LINE('Project Title: ' || proj_rec.proj_title);
    END LOOP;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No projects found.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;
/
```

Diagram:

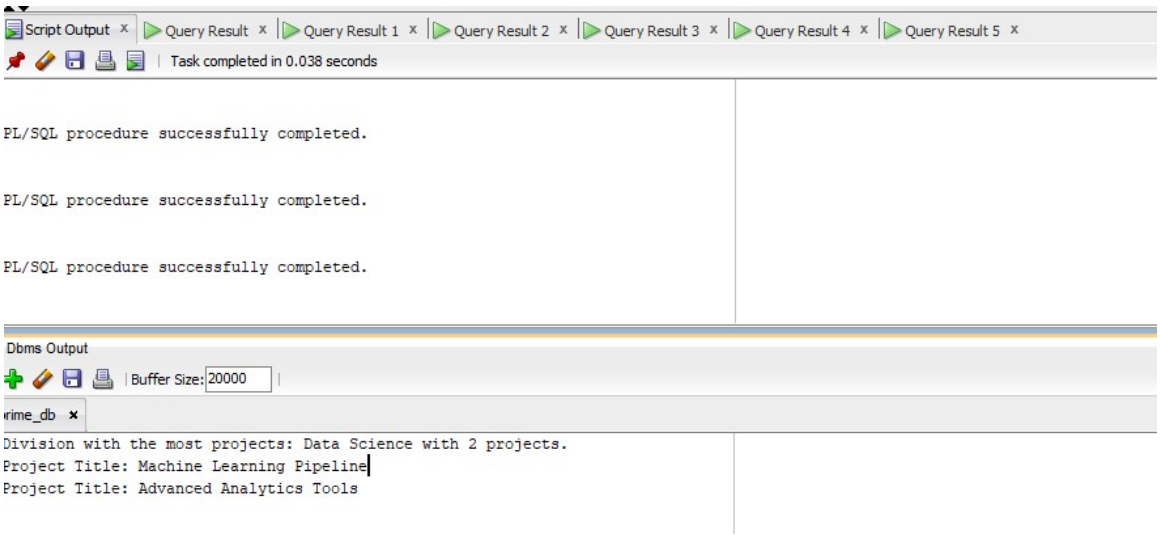


Figure 10: Most Frequent Division in Projects

Query 11: Trigger on Employee Update

Description:
This PL/SQL trigger displays all details of an employee whenever their salary is updated in the employee table.

Code:

```
CREATE OR REPLACE TRIGGER trg_employee_salary_update
AFTER UPDATE OF annual_salary ON employee
FOR EACH ROW
BEGIN
    IF :OLD.annual_salary <> :NEW.annual_salary THEN
        DBMS_OUTPUT.PUT_LINE('Employee ID: ' || :NEW.ID);
        DBMS_OUTPUT.PUT_LINE('Employee Name: ' || :NEW.emp_name);
        DBMS_OUTPUT.PUT_LINE('Division Name: ' || :NEW.division_name);
        DBMS_OUTPUT.PUT_LINE('Updated Salary: ' || :NEW.annual_salary);
        DBMS_OUTPUT.PUT_LINE('-------------------------------------');
    END IF;
END;
/
--checking triggered output
UPDATE employee
SET annual_salary = 80000
WHERE ID = 101;
```

Diagram:

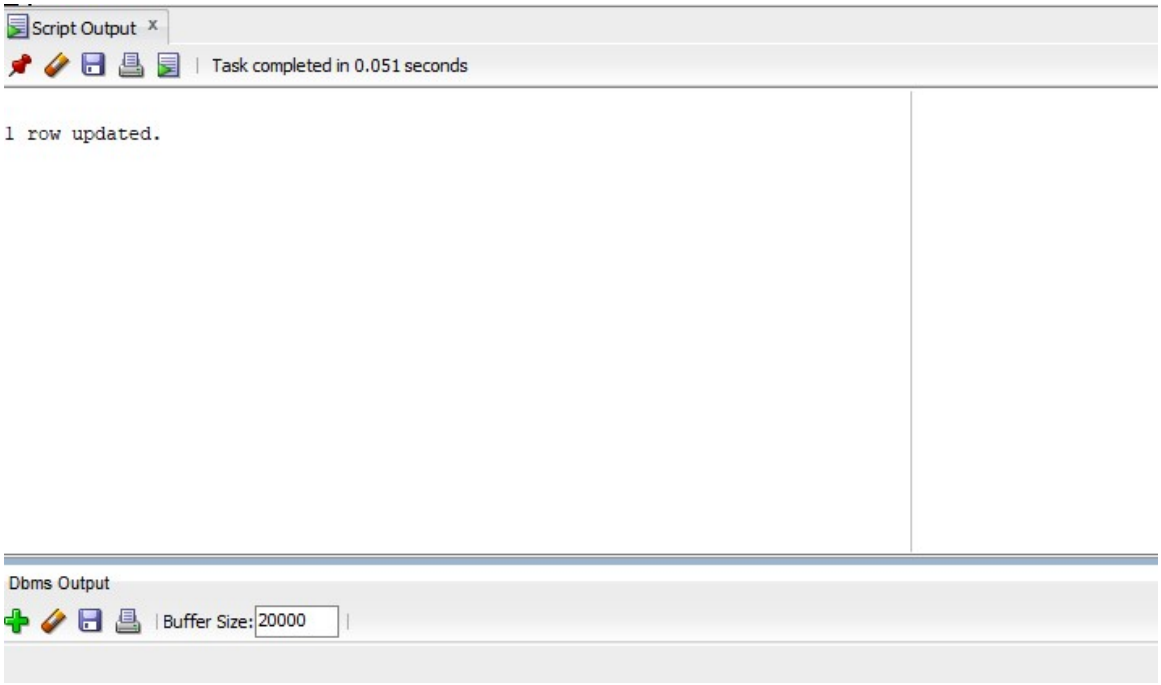


Figure 11: Trigger on Employee Salary Update

Query 12: Insert Projects Using Cursor

Description:
This PL/SQL block inserts two new projects in the project table and then uses a CURSOR to enter a new record in the dependency table for projects without dependencies.

Code:

```
DECLARE
    -- Cursor to select projects that do not have dependencies
    CURSOR project_cursor IS
        SELECT p.proj_id, p.proj_title
        FROM project p
        LEFT JOIN dependency d ON p.proj_id = d.proj_id
        WHERE d.proj_id IS NULL;  -- Projects without dependencies

    v_proj_id project.proj_id%TYPE;
    v_proj_title project.proj_title%TYPE;

BEGIN
    INSERT INTO project (proj_id, proj_title, division_name, duration_months)
    VALUES (2001, 'Cyber Security', 'IT Division', 12);
    INSERT INTO project (proj_id, proj_title, division_name, duration_months)
    VALUES (2002, 'Cloud Migration', 'IT Division', 6);
    FOR project_rec IN project_cursor LOOP
        v_proj_id := project_rec.proj_id;
        v_proj_title := project_rec.proj_title;

        -- Only insert a dependency if the project exists in the project table
        INSERT INTO dependency (proj_id, dependent_proj_id)
        VALUES (v_proj_id, v_proj_id);  -- Here you might want to specify a real dependent_proj_id
    END LOOP;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Projects inserted and dependencies updated successfully.');

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;

SELECT proj_id, proj_title FROM project;
```

Diagram:

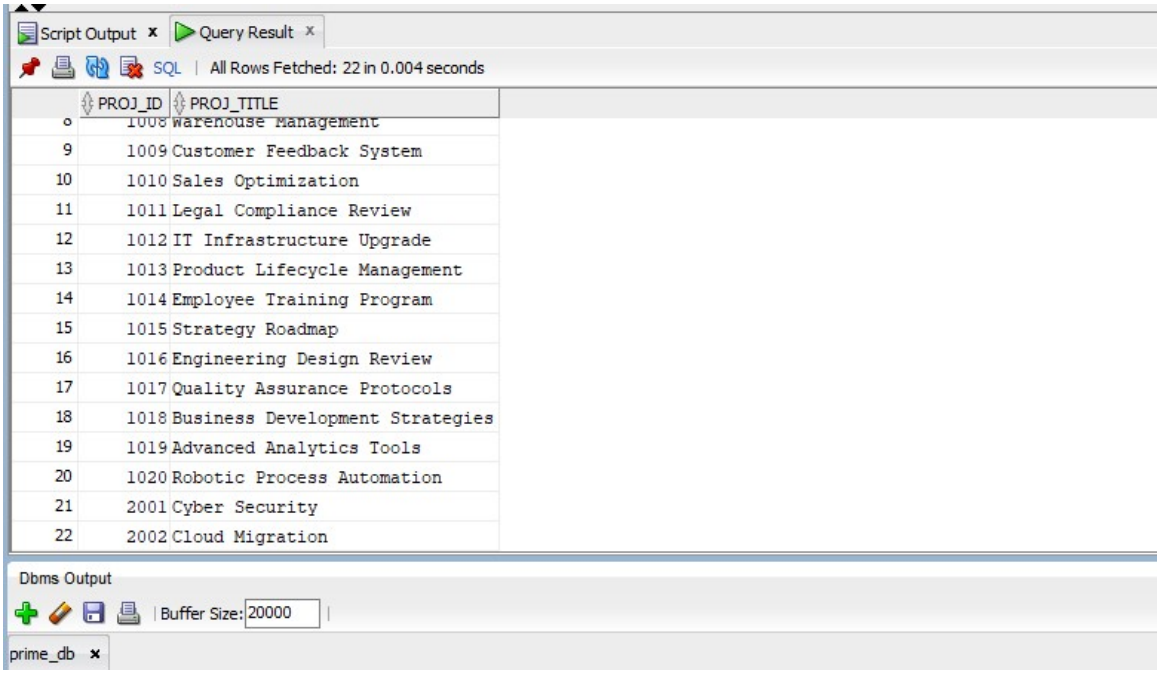


Figure 12: Insert Projects Using Cursor

2 Conclusion

This report demonstrates a variety of PL-SQL queries used to analyze and manage database information. Each query showcases different PL-SQL features and their practical applications in database management.